

Customer Journey Map

•

Date	4 July 2026
Team ID	XXXXXX
Project Name	A Comprehensive Measure of Well Being
Maximum Marks	3 Marks

HDI Predictor: Machine Learning Powered Flask Web Application

A professional, interactive, and responsive web application that utilizes Machine Learning to predict the **Human Development Index (HDI)** based on a country's development indicators.

Key Features

- **ML Predictions:** Predicts the Human Development Index (\$HDI\$ score) based on:
 - Life Expectancy at Birth (Health Dimension)
 - Expected Years of Schooling (Education Input Dimension)
 - Mean Years of Schooling (Education Output Dimension)
 - Gross National Income (GNI) Per Capita (PPP \$) (Decent Standard of Living Dimension)
- **Interactive Dashboard:** Offers global dataset metrics, dataset characteristics summary, and inline data plots.
- **Country Preference Prefill:** Choose from 36 countries to automatically load their latest real-world metrics into the prediction form.

- **Exploratory Data Analysis (EDA) Viewer:** Tabbed container displaying Correlation Heatmap, Feature Distributions, and GNI Scatter visualizations.
- **Custom Prediction Result Details:** Visualized category classification, dimensional progress bars, policy recommendations, and download parameters.
- **Client-side Prediction History:** Locally saved historical queries within browser caching (localStorage) supporting CSV export and record cleaning.
- **Modern Interface:** High-fidelity Glassmorphic design with CSS customized Light/Dark theme switching.
- **PDF Report Exports:** Built-in print styling optimization supporting clean page formatting for document saving.

🔧 Tech Stack

- **Language:** Python 3
- **Web Framework:** Flask
- **Data Processing & ML:** NumPy, Pandas, Scikit-Learn (Linear Regression), Joblib
- **Data Visualizations:** Matplotlib, Seaborn
- **Frontend Structure:** HTML5, CSS3, JavaScript, Bootstrap 5, Font Awesome Icons

📁 Project Structure

```
hdi_predictor/
|
├── dataset/                # Dataset directory
|   ├── hdi_raw.csv         # Raw downloaded time-series indices
|   └── hdi_processed.csv   # Preprocessed panel format dataset
|
├── models/                # Serialized ML assets
|   ├── hdi_model.pkl       # Trained Scikit-Learn Linear Regression model
|   ├── scaler.pkl          # StandardScaler binary
|   └── evaluation_report.txt # Model parameters and coefficient list
|
├── static/                # Static web assets
|   ├── css/
|       └── style.css       # Modern Glassmorphic CSS with Light/Dark support
|   └── js/
```

```
| | └─ script.js          # Theme switcher, syncing sliders, and cache history
| | └─ plots/             # Matplotlib / Seaborn visualization plots
| |   └─ correlation_matrix.png
| |   └─ hdi_distribution.png
| |   └─ gni_vs_hdi_scatter.png
| |   └─ ...
|
| └─ templates/           # Jinja2 HTML Templates
|   └─ base.html          # Base shell (navigation, sidebar, loading overlay)
|   └─ index.html         # Main dashboard and input form
|   └─ result.html        # Prediction score, classification, and guidelines
|   └─ about.html         # Mathematical formulas and model coefficients
|   └─ 404.html           # Custom page not found error template
|   └─ 500.html           # Custom internal server error template
|
| └─ tests/               # Testing suite package
|   └─ __init__.py
|   └─ test_app.py        # Routes, handlers, and form validations tests
|   └─ test_model.py      # Preprocessing pipeline and ML predictions tests
|
| └─ app.py               # Flask server entrypoint
| └─ config.py            # Application configuration settings
| └─ train_model.py       # ML pipeline (download, clean, plot, train, serialize)
| └─ requirements.txt     # Python packages list
| └─ .gitignore           # Git exclude file
```

Local Installation & Setup

Follow these steps to run the application locally on your Windows environment.

1. Clone or Copy the Repository

Place the project directory files inside your chosen workspace folder:

```
cd C:\Users\User\.gemini\antigravity\scratch\hdi_predictor
```

2. Set Up a Virtual Environment (Optional but Recommended)

Create and activate a virtual environment to isolate python dependencies:

```
# Create venv
python -m venv venv
```

```
# Activate venv on Windows PowerShell
.\venv\Scripts\Activate.ps1
```

3. Install Dependencies

Install all required libraries using pip:

```
python -m pip install -r requirements.txt
```

4. Execute the ML Pipeline

Run the preprocessing and training script to download raw UN records, generate EDA plots, and fit the regression model:

```
python train_model.py
```

5. Launch the Web Application

Start the Flask dev server:

```
python app.py
```

The application will launch on your local host: <http://127.0.0.1:5000/>. Open it in any standard browser to explore the dashboard.



Running Unit Tests

Run the test suite containing model fitting assertions and server routing checks:

```
python -m unittest discover -s tests
```



Model Performance Characteristics

Our trained Linear Regression model performs with the following statistical traits on unseen test indicators (80-20 partition): * **R-squared Coefficient (R^2):** 99.47% (Excellent fit relative to the mathematical index) * **Mean Absolute Error (MAE):** \$0.0098\$ * **Root Mean Squared Error (RMSE):** \$0.0122\$

